Structs (estruturas)

Prof. Thiago Felski Pereira, MSc.
Adaptado: Elisangela Maschio de Miranda

Conceito

- Struct é um tipo de dado criado pelo próprio programador, que agrupa várias variáveis — de tipos diferentes — dentro de uma única variável.
- *Pense nela como uma ficha: um registro só, que guarda vários campos juntos, cada um com o seu próprio tipo.*

UNIVALI

O problema que a struct resolve

- Imagine que você precisa guardar o nome, a nota e a situação da mensalidade de vários alunos.
- Sem struct, seria necessária uma variável separada para cada informação, de cada aluno:

```
string nomeAluno1;
float notaAluno1;
bool mensalidadePagaAluno1;

string nomeAluno2;
float notaAluno2;
bool mensalidadePagaAluno2;
// ...e assim por diante, para cada aluno
```

- **Isso cresce rápido, é fácil de errar (nome trocado, variável esquecida) e não escala para muitos alunos.**

UNIVALI

Antes de ver a sintaxe

ATIVIDADE 1

- **Pensando no problema anterior: quais campos vocês colocariam numa "ficha de aluno"?**
- Listem os campos e o tipo de cada um. Por exemplo: nome — texto.
- *Não se preocupem com a sintaxe ainda — o objetivo é só organizar a ideia.*

UNIVALI

Agrupando os dados

- Cada campo que vocês listaram vira um membro dentro da struct.

```
struct Jogador {
    string nomeDoJogador;
    int forca, destreza, inteligencia;
};
```

- Sem struct, seria necessário repetir as variáveis para cada jogador (nomeDoJogador1, forca1, ... nomeDoJogador2, forca2, ...).
- **Com struct, agrupamos tudo isso em um único tipo — e usamos só o nome da struct na chamada.**

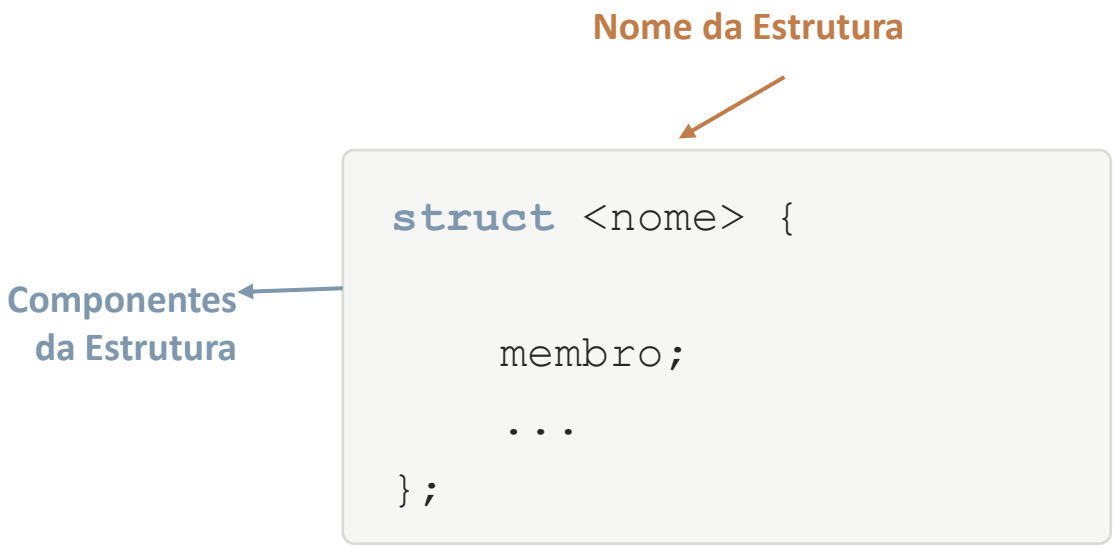
UNIVALI

Sintaxe

- Para criar uma estrutura de dados é utilizada a palavra reservada struct.
- Toda estrutura deve ser criada antes de qualquer função — inclusive antes do main.
- Toda estrutura tem um nome, e seus membros são declarados dentro de um bloco { }.
- Termina-se a declaração com ponto e vírgula (;).

UNIVALI

Sintaxe




```
struct produto {
    int cod;
    string nome;
};
```


UNIVALI

Sintaxe: exemplo

```
struct Mercado {
    int codigo;
    string nomeDoProduto;
    float preco;
};

int main () {
    Mercado prateleira1;
    return 0;
}
```

- A struct Mercado contém todas as variáveis referentes às informações de um produto.
- Para utilizá-la, é necessário criar uma variável do "tipo" Mercado — como em Mercado prateleira1;

UNIVALI

Acesso aos membros da Estrutura

```
Mercado prateleira1;

cout << "Codigo: ";
cin >> prateleira1.codigo;
cout << "Preco: ";
cin >> prateleira1.preco;
```

- Para acessar um membro da struct, usa-se o nome da variável, um ponto, e o nome do membro:
  - **variavel.membro (ex.: prateleira1.preco)**

UNIVALI

Agora, a struct Aluno

ATIVIDADE 2

- **Usando os campos que vocês listaram na Atividade 1, declarem a struct Aluno.**
- **Depois, escrevam o código que lê os dados de um aluno pelo teclado (nome, nota, mensalidade).**
- *Dica: sigam o mesmo padrão do exemplo Mercado — struct primeiro, depois a variável, depois o cin.*

UNIVALI

Escopo: struct local x global

- **Uma struct pode ser local ou global.**
  - Local: válida somente na função onde foi declarada.
  - Global: válida em todas as funções abaixo de sua declaração.
- Assim como em variáveis comuns, o mais comum é declarar a struct antes do main, como global — para que qualquer função do programa possa usá-la.

UNIVALI

Atribuição entre Structs

- Uma struct pode ser atribuída diretamente a outra do mesmo tipo — todos os membros são copiados de uma vez.

```
Mercado prateleira1 = {1, "Biscoitos", 4.50};
Mercado prateleira2 = prateleira1;

cout << prateleira2.nomeDoProduto; // Biscoitos
```

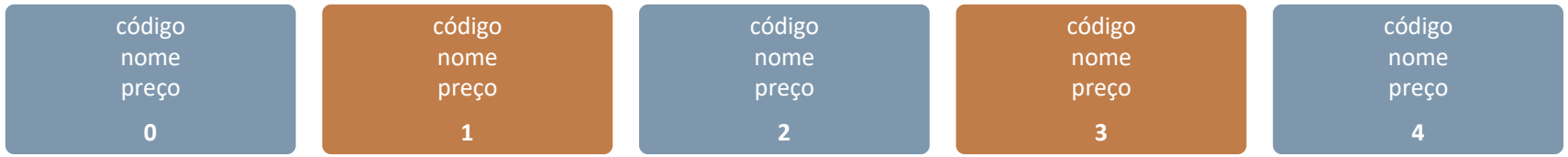

UNIVALI

Vetores de Structs

- É possível criar vetores cujos elementos são estruturas — cada posição do vetor guarda um conjunto completo de membros.

```
Mercado produtos[TAM];

for (int i = 0; i < TAM; i++) {
    cin >> produtos[i].codigo;
    cin >> produtos[i].preco;
}
```



UNIVALI

Vetor do tipo struct × Vetor dentro da struct

```
struct Mercado {
    int codigo;
    string nomeDoProduto;
    float preco;
};

Mercado produtos[TAM];
```

- Cada índice do vetor é um Mercado inteiro — códigos, nomes e preços diferentes por produto.

```
struct Mercado {
    int codigo;
    string nomeDoProduto;
    float preco[TAM];
};

Mercado produtos;
```

- É um único Mercado, com um vetor de preços dentro dele — mesmo código e nome, vários preços.

UNIVALI

Matrizes de Structs

- A mesma lógica vale para matrizes: cada posição [i][j] guarda um conjunto completo de membros da struct.

```
Mercado produtos[TAM][TAM];

for (int i = 0; i < TAM; i++) {
    for (int j = 0; j < TAM; j++) {
        cin >> produtos[i][j].codigo;
    }
}
```


UNIVALI

Estruturas Aninhadas

- Uma struct pode conter outra struct como membro — isso se chama estrutura aninhada.

```
struct Data {
    int dia, mes, ano;
};

struct Produto {
    int codigo;
    Data entrega;
    Data validade;
};
```

- Para acessar um membro de uma struct aninhada, encadeiam-se os pontos: variavel.membroStruct.membroInterno
  - **produto1.entrega.dia** • **produto1.validade.dia**

UNIVALI

Passagem de Parâmetro e Retorno de Função

- Uma struct pode ser passada por parâmetro para uma função — por valor ou por referência, exatamente como já vimos com outros tipos.
- Uma função também pode retornar uma struct.

```
struct Teste { int a, b, res; };

Teste dobrar (Teste &y) {
    y.a = y.a + y.b;
    return y;
}
```


UNIVALI

Conectando com o que já vimos

- Passar uma struct por parâmetro segue exatamente as mesmas regras de valor/referência que já vimos em sub-rotinas — só que agora copiamos (ou referenciamos) todos os membros de uma vez, não um valor só.
- Um vetor de structs combina duas ideias que vocês já dominam: vetor e struct — cada posição é só mais uma variável do tipo struct.
- **Em breve, com ponteiros, vocês vão conseguir apontar diretamente para uma struct — é o que torna possível, por exemplo, montar listas ligadas.**

UNIVALI

Resumo: usando structs

1. struct <Nome> { membros; }; define um novo tipo.
2. Declare uma variável desse tipo para poder usá-la: Nome variavel;
3. Acesse um membro com variavel.membro
4. Uma struct pode conter vetores, matrizes, e até outras structs (aninhadas).
5. Passagem por valor/referência e retorno funcionam como em qualquer outro tipo.

UNIVALI

Obrigado pela atenção

contato: Felski@univali.br